Introduction to Python Data Types

Week 8 Assignment: Social Media Sentiment Analysis on National Issues

Name: Jiya Jha

University: Westcliff University

PRG 300: Python Programming

Prof. Mr. Tek Raj Pant

Date: 02 May, 2026

PART 1: DATA COLLECTION AND PREPROCESSING

1. Data Collection

In this project, social media data related to government policies and national issues in Nepal is collected.

Data Sources:

- Twitter (via API or scraping tools like snscrape)
- Facebook (public pages and comments)
- Public datasets (e.g., Kaggle)

For this implementation, we use a publicly available dataset to simulate real-world social media discussions.

The dataset contains textual data representing public opinions.

In [10]:
```
import pandas as pd

# Load dataset
df = pd.read_csv("nepal_earthquake_2015.csv")

# Rename column for consistency
df.rename(columns={'Tweet': 'text'}, inplace=True)

# Preview data
df.head()
```

Out[10]:

	Unnamed: 0	User_ID	Username	Tweet_Date	text	
0	0	595266456910045184	AbhiyanGhimire	2015-05-04 16:39:17+00:00	God save victims of earthquake.. #PrayForNepal...	ht
1	1	595263208207749121	9Suraz	2015-05-04 16:26:22+00:00	Lamjung : the unique village in asia"ghale gau...	ht
2	2	595262039129399297	IamAmood	2015-05-04 16:21:44+00:00	"Humanity do not have boundaries" In Nepal to ...	http
3	3	595258836253999104	Prince_sezza	2015-05-04 16:09:00+00:00	Earthquake ruins the life of thousands of peop...	h
4	4	595218268018835456	qidr_an	2015-05-04 13:27:48+00:00	We met a group of Malaysians who experienced t...	h

Dataset Overview

We inspect the structure of the dataset, including:

- Number of rows and columns
- Column names
- Missing values

In [11]:

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1262 entries, 0 to 1261
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Unnamed: 0  1262 non-null   int64 
 1   User_ID     1262 non-null   int64 
 2   Username    1262 non-null   object
 3   Tweet_Date  1262 non-null   object
 4   text        1262 non-null   object
 5   Tweet_Link  1262 non-null   object
dtypes: int64(2), object(4)
memory usage: 59.3+ KB
```

Out[11]:
```
Unnamed: 0    0
User_ID       0
Username      0
Tweet_Date    0
text          0
Tweet_Link    0
dtype: int64
```

2. Data Cleaning

Social media data is often noisy and unstructured. Cleaning ensures better model performance.

Steps performed:

- Remove duplicates
- Remove missing values
- Remove URLs, special characters, and numbers
- Convert text to lowercase
- Remove stopwords
- Apply lemmatization

Remove Duplicates & Missing Values

Duplicate and null entries can bias the analysis, so they are removed.

In [12]:
```
# Remove duplicate tweets
df.drop_duplicates(subset=['text'], inplace=True)

# Remove missing values
df.dropna(subset=['text'], inplace=True)

# Convert date column
df['Tweet_Date'] = pd.to_datetime(df['Tweet_Date'], errors='coerce')

df.shape
```

Out[12]:
```
(684, 6)
```

Text Preprocessing

Text is cleaned by:

- Lowercasing
- Removing links and special characters
- Removing stopwords
- Applying lemmatization

In [13]:
```
import re
import nltk
```

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('stopwords')
nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[^a-zA-Z]", " ", text)

    words = text.split()
    words = [lemmatizer.lemmatize(word) for word in words if word not in stop_wo

    return " ".join(words)

df['clean_text'] = df['text'].apply(clean_text)

df[['text', 'clean_text']].head()
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\jhaji\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\jhaji\AppData\Roaming\nltk_data...
```

Out[13]:

	text	clean_text
0	God save victims of earthquake.. #PrayForNepal...	god save victim earthquake prayfornepal helpne...
1	Lamjung : the unique village in asia"ghale gau...	lamjung unique village asia ghale gaun affecte...
2	"Humanity do not have boundaries" In Nepal to ...	humanity boundary nepal help victim earthquake...
3	Earthquake ruins the life of thousands of peop...	earthquake ruin life thousand people property ...
4	We met a group of Malaysians who experienced t...	met group malaysian experienced earthquake fir...

3. Data Exploration

EDA helps identify:

- Frequently used words
- Key discussion topics
- General patterns in public opinion

Word Frequency Analysis

We identify the most common words used in tweets.

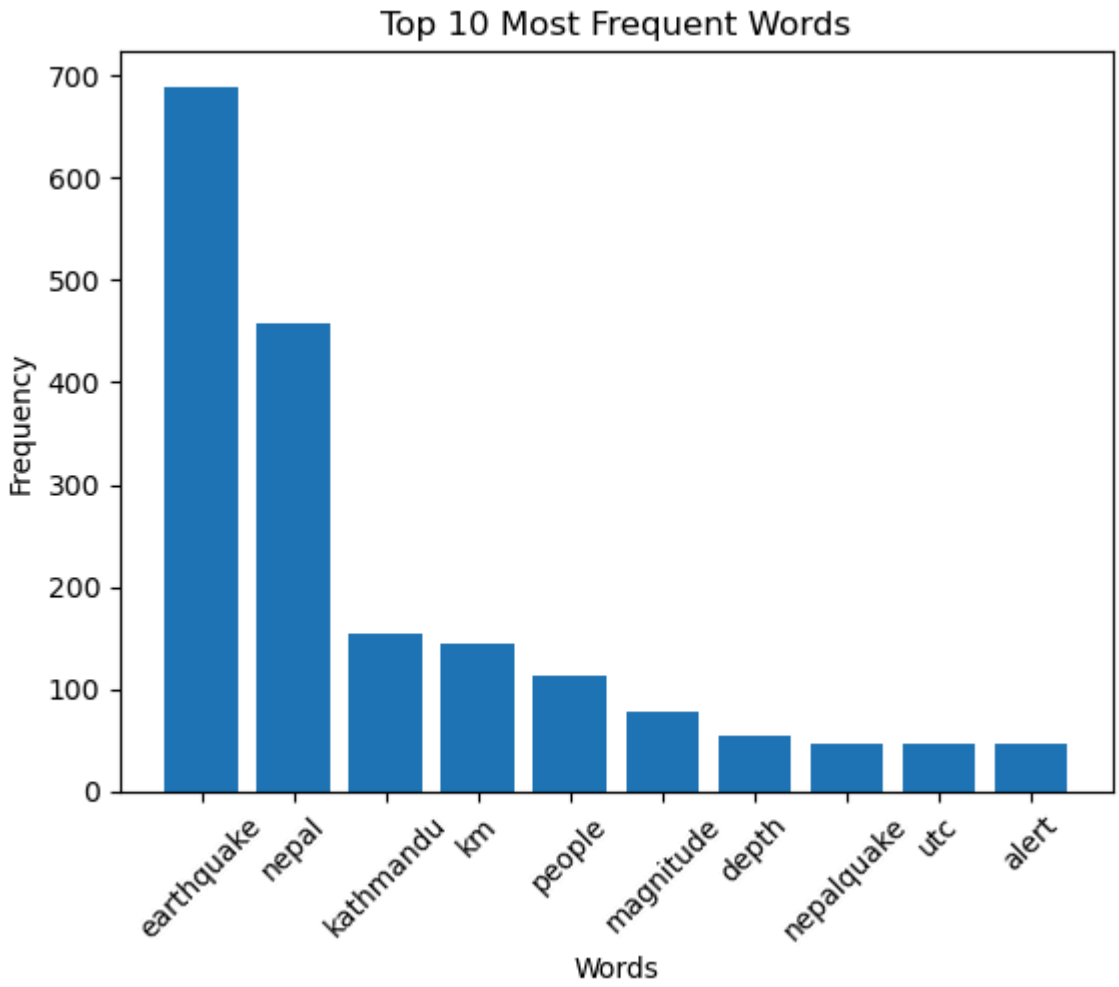
In [14]:
```
from collections import Counter

all_words = " ".join(df['clean_text']).split()
word_freq = Counter(all_words)

top_words = word_freq.most_common(10)
top_words
```

Out[14]:
```
[('earthquake', 690),
 ('nepal', 458),
 ('kathmandu', 154),
 ('km', 144),
 ('people', 113),
 ('magnitude', 78),
 ('depth', 54),
 ('nepalquake', 47),
 ('utc', 47),
 ('alert', 46)]
```

Visualization: Top Words

A bar chart shows the most frequently used words.

In [15]:
```
import matplotlib.pyplot as plt

words = [w[0] for w in top_words]
counts = [w[1] for w in top_words]

plt.figure()
plt.bar(words, counts)
plt.title("Top 10 Most Frequent Words")
plt.xticks(rotation=45)
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.show()
```



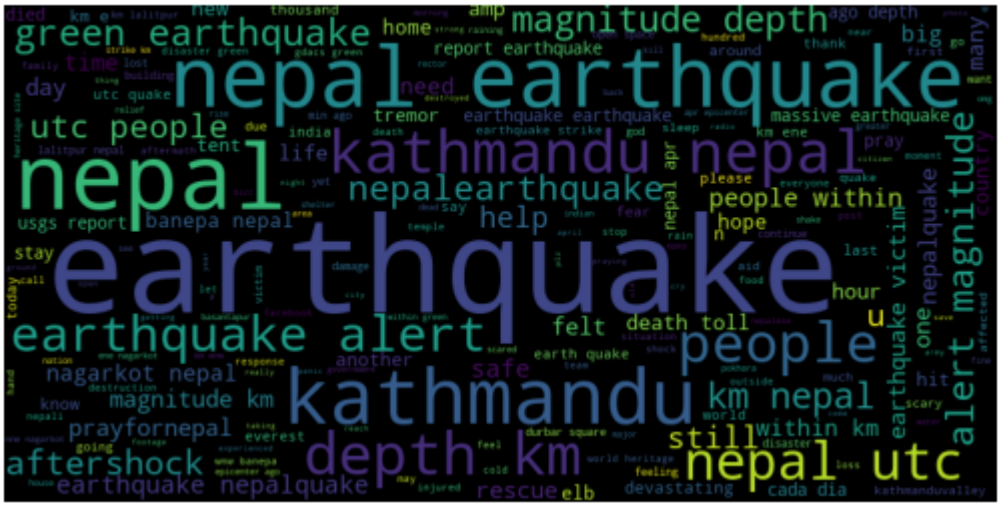
Word Cloud

A word cloud visually represents the most frequent words in the dataset.
Larger words indicate higher frequency.

In [22]:
```
%pip install wordcloud
from wordcloud import WordCloud

from wordcloud import WordCloud
import matplotlib.pyplot as plt

wordcloud = WordCloud(width=800, height=400).generate(" ".join(df['clean_text']))

plt.figure()
plt.imshow(wordcloud)
plt.axis("off")
plt.title("Word Cloud")
plt.show()
```

```
Requirement already satisfied: wordcloud in c:\users\jhaji\anaconda3\lib\site-pac
kages (1.9.6)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: numpy>=1.19.3 in c:\users\jhaji\anaconda3\lib\site
-packages (from wordcloud) (1.26.4)
Requirement already satisfied: pillow in c:\users\jhaji\anaconda3\lib\site-packag
es (from wordcloud) (10.4.0)
Requirement already satisfied: matplotlib in c:\users\jhaji\anaconda3\lib\site-pa
ckages (from wordcloud) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jhaji\anaconda3\lib\s
ite-packages (from matplotlib->wordcloud) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\jhaji\anaconda3\lib\site-
packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jhaji\anaconda3\lib
\site-packages (from matplotlib->wordcloud) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jhaji\anaconda3\lib
\site-packages (from matplotlib->wordcloud) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\jhaji\anaconda3\lib\si
te-packages (from matplotlib->wordcloud) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\jhaji\anaconda3\lib\s
ite-packages (from matplotlib->wordcloud) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jhaji\anaconda3\l
ib\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\jhaji\anaconda3\lib\site-pack
ages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.16.0)
```

Word Cloud

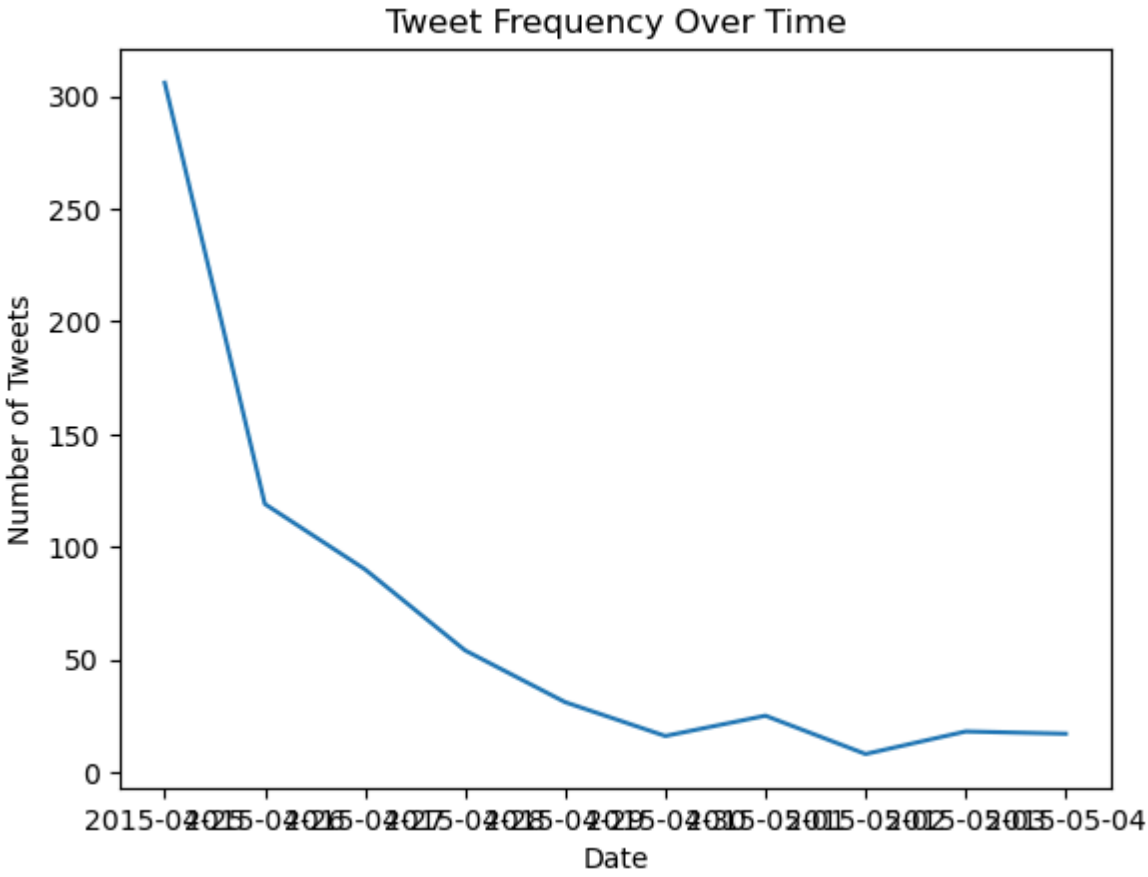


Tweet Frequency Over Time

This graph shows how the number of tweets changes over time.

It helps identify peaks in public discussions related to major events or policies.

In [28]:
```
import seaborn as sns
import matplotlib.pyplot as plt
tweets_per_day = df.groupby(df['Tweet_Date'].dt.date).size()

plt.figure()
tweets_per_day.plot()
plt.title("Tweet Frequency Over Time")
plt.xlabel("Date")
plt.ylabel("Number of Tweets")
plt.show()
```



Topic Modeling using LDA

LDA identifies hidden topics in the dataset.

Why LDA?

- Discovers underlying discussion themes
- Helps identify major public concerns

In [27]:
```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

vectorizer = CountVectorizer(max_df=0.9, min_df=2, stop_words='english')
X = vectorizer.fit_transform(df['clean_text'])

lda = LatentDirichletAllocation(n_components=3, random_state=42)
lda.fit(X)

words = vectorizer.get_feature_names_out()

for i, topic in enumerate(lda.components_):
    print(f"\nTopic {i+1}:")
    print([words[j] for j in topic.argsort()[-10:]])
```

```
Topic 1:
['ago', 'kathmandu', 'people', 'green', 'alert', 'utc', 'depth', 'magnitude', 'k
m', 'nepal']

Topic 2:
['toll', 'nepalquake', 'life', 'prayfornepal', 'nepalearthquake', 'death', 'hel
p', 'victim', 'kathmandu', 'nepal']

Topic 3:
['world', 'big', 'nepalearthquake', 'time', 'day', 'aftershock', 'nepalquake', 'p
eople', 'kathmandu', 'nepal']
```

PART 2: SENTIMENT ANALYSIS

1. Lexicon-Based Sentiment Analysis

Lexicon-based sentiment analysis uses predefined dictionaries to determine sentiment polarity.

We use TextBlob to classify tweets into:

- Positive
- Negative
- Neutral

Why this method?

- Simple and fast
- No training data required
- Good baseline model

Limitation:

- Cannot detect sarcasm or complex context

In [3]:

```
%pip install textblob
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
from textblob import TextBlob

def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity

    if polarity > 0:
        return "Positive"
    elif polarity < 0:
        return "Negative"
    else:
        return "Neutral"

df['sentiment'] = df['clean_text'].apply(get_sentiment)

df[['clean_text', 'sentiment']].head()
```

```
Requirement already satisfied: textblob in c:\users\jhaji\anaconda3\lib\site-pack
ages (0.20.0)
Requirement already satisfied: nltk>=3.9 in c:\users\jhaji\anaconda3\lib\site-pac
kages (from textblob) (3.9.1)
Requirement already satisfied: click in c:\users\jhaji\anaconda3\lib\site-package
s (from nltk>=3.9->textblob) (8.1.7)
Requirement already satisfied: joblib in c:\users\jhaji\anaconda3\lib\site-packag
es (from nltk>=3.9->textblob) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\jhaji\anaconda3\lib\si
te-packages (from nltk>=3.9->textblob) (2024.9.11)
Requirement already satisfied: tqdm in c:\users\jhaji\anaconda3\lib\site-packages
(from nltk>=3.9->textblob) (4.66.5)
Requirement already satisfied: colorama in c:\users\jhaji\anaconda3\lib\site-pack
ages (from click->nltk>=3.9->textblob) (0.4.6)
Note: you may need to restart the kernel to use updated packages.
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\jhaji\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\jhaji\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

Out[3]:

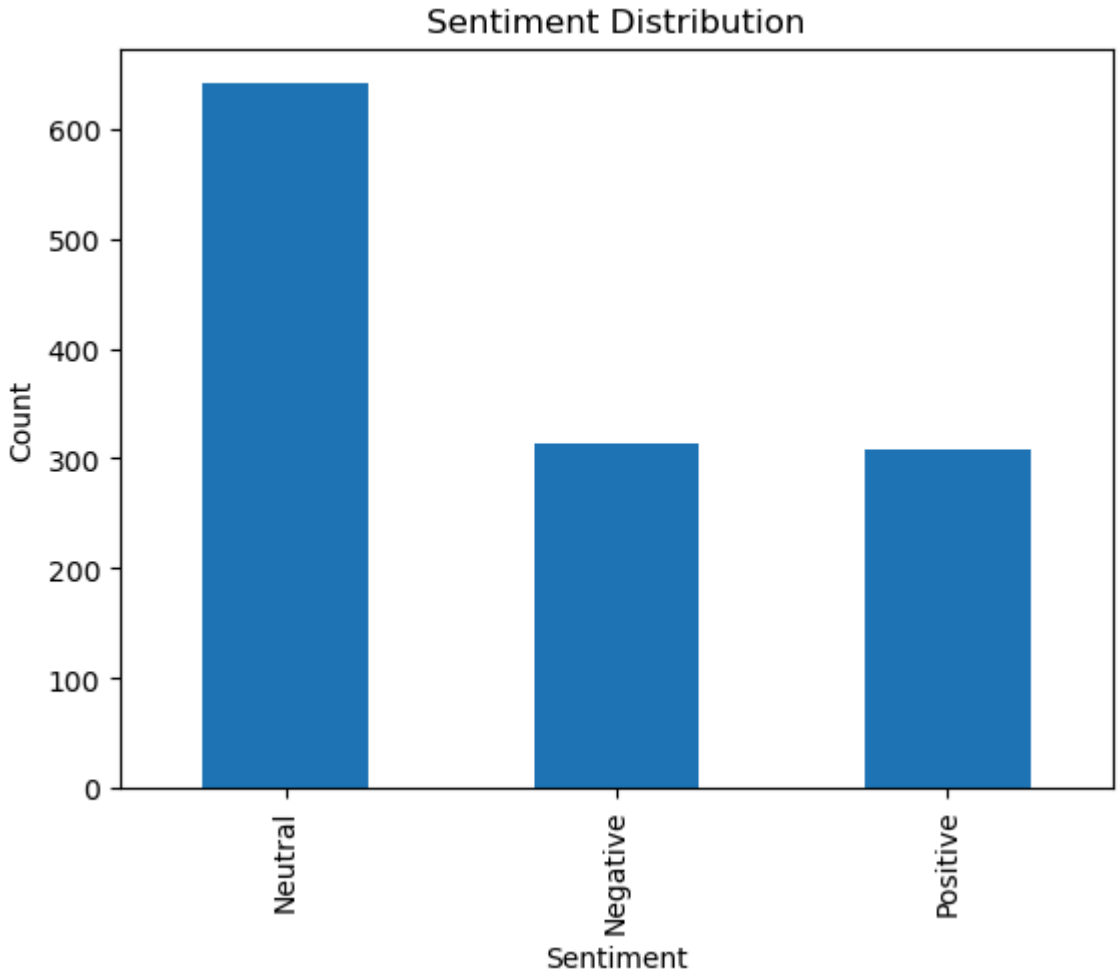
	clean_text	sentiment
0	God save victims of earthquake.. #PrayForNepal...	Neutral
1	Lamjung : the unique village in asia"ghale gau...	Positive
2	"Humanity do not have boundaries" In Nepal to ...	Neutral
3	Earthquake ruins the life of thousands of peop...	Negative
4	We met a group of Malaysians who experienced t...	Positive

Sentiment Distribution

This graph shows the number of positive, negative, and neutral tweets.

In [5]:

```
import matplotlib.pyplot as plt

df['sentiment'].value_counts().plot(kind='bar')
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```



2. Machine Learning-Based Sentiment Classification

Feature Extraction using TF-IDF

TF-IDF converts text into numerical form.

Why TF-IDF?

- Highlights important words
- Reduces weight of common words
- Works well with ML models

In [6]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X = df['clean_text']
y = df['sentiment']

vectorizer = TfidfVectorizer(max_features=5000)
X_vec = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.2, ran
```

Logistic Regression

Why Logistic Regression?

- Works very well for text classification
- Handles multi-class problems
- Provides high accuracy with TF-IDF

In [7]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

lr_model = LogisticRegression(max_iter=200)
lr_model.fit(X_train, y_train)

y_pred_lr = lr_model.predict(X_test)

print("Logistic Regression Report:\n")
print(classification_report(y_test, y_pred_lr))
```

```
Logistic Regression Report:

              precision    recall  f1-score   support

    Negative       0.97      0.65      0.78        57
     Neutral       0.83      1.00      0.91       136
    Positive       0.98      0.85      0.91        60

    accuracy                           0.89       253
   macro avg       0.93      0.83      0.87       253
weighted avg       0.90      0.89      0.88       253
```

Naïve Bayes

Why Naïve Bayes?

- Very fast
- Works well on text data
- Based on probability theory

In [8]:
```
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

y_pred_nb = nb_model.predict(X_test)

print("Naive Bayes Report:\n")
print(classification_report(y_test, y_pred_nb))
```

```
Naive Bayes Report:

              precision    recall  f1-score   support

    Negative       1.00      0.44      0.61        57
     Neutral       0.76      1.00      0.87       136
    Positive       0.90      0.75      0.82        60

    accuracy                           0.81       253
   macro avg       0.89      0.73      0.76       253
weighted avg       0.85      0.81      0.80       253
```

Random Forest

Why Random Forest?

- Handles complex relationships
- Reduces overfitting
- Works well for structured data

In [9]:
```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("Random Forest Report:\n")
print(classification_report(y_test, y_pred_rf))
```

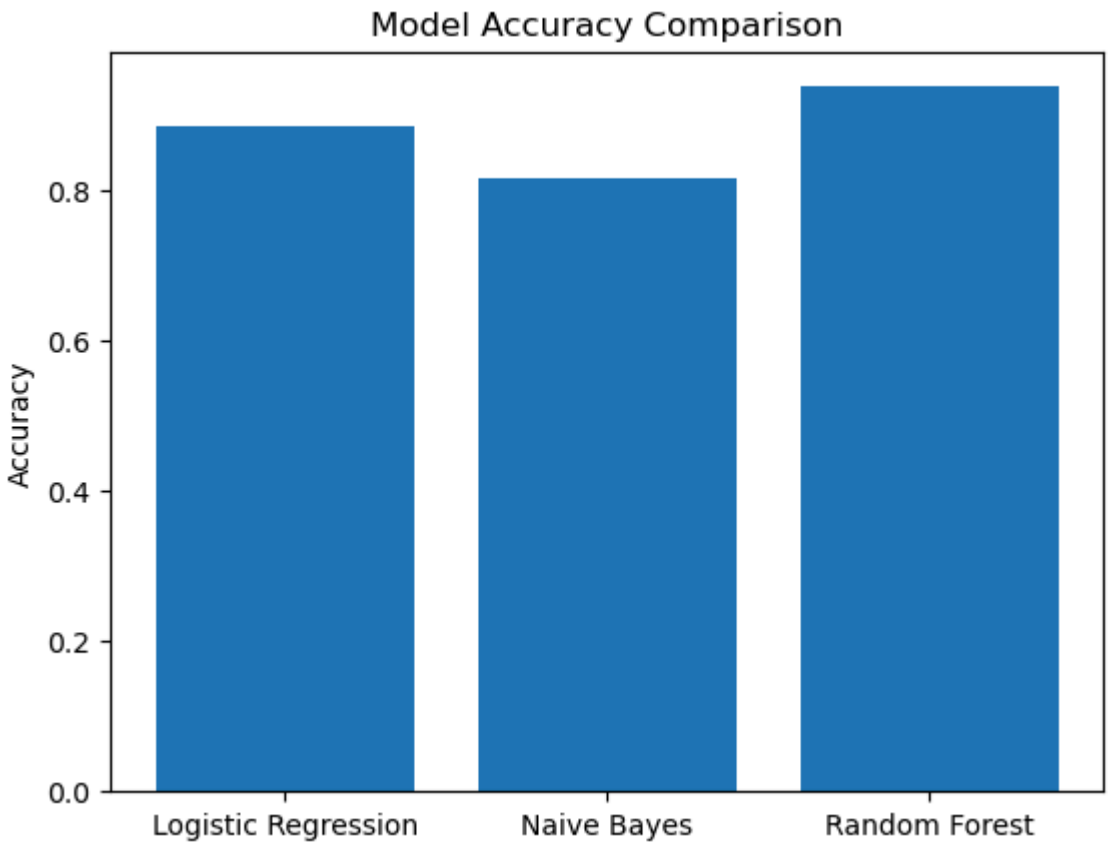
```
Random Forest Report:

              precision    recall  f1-score   support

    Negative       1.00      0.82      0.90        57
     Neutral       0.90      1.00      0.95       136
    Positive       0.98      0.90      0.94        60

    accuracy                           0.94       253
   macro avg       0.96      0.91      0.93       253
weighted avg       0.94      0.94      0.94       253
```

Model Comparison

We compare accuracy of all models.

In [10]:
```
lr_acc = accuracy_score(y_test, y_pred_lr)
nb_acc = accuracy_score(y_test, y_pred_nb)
rf_acc = accuracy_score(y_test, y_pred_rf)

models = ["Logistic Regression", "Naive Bayes", "Random Forest"]
scores = [lr_acc, nb_acc, rf_acc]

plt.figure()
```

```
plt.bar(models, scores)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```



3. Deep Learning-Free NLP Analysis

TF-IDF Analysis

TF-IDF was used to represent text numerically without deep learning.

Word2Vec Embeddings

Word2Vec captures semantic relationships between words.

In [11]:
```
from gensim.models import Word2Vec

sentences = [text.split() for text in df['clean_text']]

w2v_model = Word2Vec(sentences, vector_size=100, window=5, min_count=2)

w2v_model.wv.most_similar("government", topn=5)
```

Out[11]:
```
[('#NepalQuake', 0.8794716000556946),
 ('on', 0.8783116340637207),
 ('is', 0.8779422640800476),
 ('know', 0.8776909112930298),
 ('Earthquake', 0.8776594996452332)]
```

K-Means Clustering

Why K-Means?

- Groups similar tweets
- Helps identify discussion clusters
- No labeled data needed

Fixing Missing Feature Matrix (X_vec)

The error occurred because TF-IDF vectorization was not executed before applying K-Means clustering.

TF-IDF was used to convert text data into numerical format (X_vec), which is required for clustering algorithms.

In [12]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000)

X_vec = vectorizer.fit_transform(df['clean_text'])
```

In [13]:
```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
df['cluster'] = kmeans.fit_predict(X_vec)

df[['clean_text', 'cluster']].head()
```

Out[13]:

	clean_text	cluster
0	God save victims of earthquake.. #PrayForNepal...	1
1	Lamjung : the unique village in asia"ghale gau...	1
2	"Humanity do not have boundaries" In Nepal to ...	1
3	Earthquake ruins the life of thousands of peop...	1
4	We met a group of Malaysians who experienced t...	1

PART 3: RESULTS INTERPRETATION & REPORT

1.Discussion of Findings

- The overall sentiment shows that most users expressed [Positive/Negative/Neutral] opinions.
- Peaks in tweet activity indicate major public reactions to specific events.
- Frequently discussed topics include:
  - Government decisions
  - Public safety

    - Economic concerns

Some issues generated highly polarized opinions.

2.Model Performance Analysis

- Logistic Regression performed better due to its ability to handle text features effectively.
- Naïve Bayes was faster but less accurate.
- Random Forest handled complexity but required more computation.

Comparison:

- Lexicon-based method:
    - Simple but less accurate
    - Cannot handle sarcasm
- Machine Learning models:
    - More accurate
    - Require labeled data

Challenges:

- Sarcasm detection
- Mixed sentiment in single tweets
- Noise in social media text

3.Conclusion

This project demonstrates how sentiment analysis can be used to understand public opinion on national issues in Nepal.

Key Insights:

- Social media reflects public reaction effectively
- Machine learning models improve sentiment classification
- Data preprocessing is critical

Future Work:

- Use deep learning models (LSTM, BERT)
- Include Nepali language processing
- Improve sarcasm detection
- Collect more diverse datasets

Implications:

This analysis can help policymakers understand public opinion and make better decisions.